

Software Design Document

Project: Home Network Solutions

Team Members: Bryan Bermudez, Joshua Revels, Brennan Ramsey

Faculty Advisor: Dr. Mohan

Date: September 2026

Table of Contents:

1. Introduction.....	2
1.1 Purpose.....	2
1.2 Scope.....	2
1.3 Intended Audience.....	2
1.4 Acronyms and Abbreviations.....	2
2. System Design.....	3
2.1 Description.....	3
2.3 User-facing systems.....	3
2.4 Non-user-facing systems.....	4
2.4.1 Web Server.....	4
2.4.2 Helper Program.....	4
2.4.3 File Synchronization.....	5
2.4.4 Password and Passkey Management.....	5
3. System Architecture.....	5
3.1 Architecture Design.....	5
3.2 Server Design.....	6
3.3 Web Application Design.....	6
3.4 Design Rationale.....	7
4. Data Design.....	7
4.1 Data Flow.....	7
4.2 Data Description.....	7

1. Introduction

1.1 Purpose

The purpose of this document is to describe and inform readers about the technical, design, and data architectures of the Home Network Solutions (HNS) project. The reasons for design choices and rationale will be explained and expanded upon. Each of the design choices will, in some way, contribute to our overarching requirements.

This project serves to facilitate the installation and maintenance of a home server on the user's home network. The project will package several tools and scripts to enable powerful features, all being managed through a web-based application. The system will focus on readability and usability as the product is aimed at non-technical users.

1.2 Scope

The scope of this project is to facilitate the installation, setup, and maintenance of a home network. Once properly installed, the finished software will have a variety of key features and self-maintenance tools. These allow the user to benefit from having a home network without the hassle of maintenance and the worry of cybersecurity attacks. The project will provide an easy-to-follow guide and documentation, as well as providing initial hardware and software to set up the system.

1.3 Intended Audience

The software's targeted demographic are non-technical users who wish to benefit from a home server of their own. Users with little technical knowledge of server-related infrastructure or terminology should still be able to utilize and set up the HNS software. These users should be able to set up and manage their own home server without a large time investment or much troubleshooting.

1.4 Acronyms and Abbreviations

Term	Definition
HNS	Home Network Solutions
VPN	Virtual Private Network
IP	Internet Protocol
OS	Operating System

API	Application Programming Interface
TCP	Transmission Control Protocol
HTTP	Hypertext Transfer Protocol
LTS	Long-Term Support
Home Server	The primary device running HNS software; the Ubuntu server.
Web Server	The service that directs connections between the web application and the client.
Web Application	Core feature of HNS software that allows users to manage HNS software and services.

2. System Design

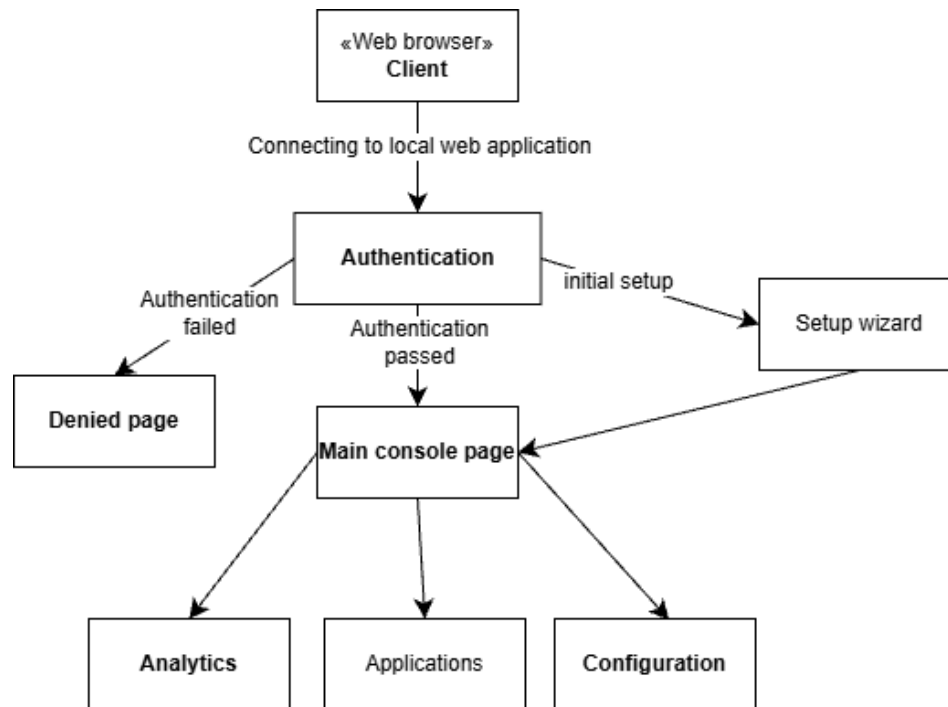
2.1 Description

A home server that connects to the local network that hosts a variety of applications and tools that are then accessed using a web application.

2.3 User-facing systems

A web-based application hosted on the home server, accessible via a web browser. The web application will have an authentication gateway that will only allow vetted devices from accessing the application. Upon access and successful authentication, the user will be taken to a main webpage with a variety of options available. Such options include the tools, applications, and analytics pages, where each option takes the user to their respective pages.

On the product's first startup, a setup wizard will be accessible through the web application. The user will be guided through general settings configurations, features to install and set up, password setup, and guide the user on next steps and how to best use the HNS software.



2.4 Non-user-facing systems

The product will be hosted on a device running an Ubuntu Server operating system. The device, known as the home server, will host a variety of tools and services.

Scripts in Bash and Python will be employed to interface the web application with tools running locally. These services and tools may run natively via the system daemon and shell, or in a Docker container.

Since some features take effect network-wide, such as Pi-hole or the VPN connection, the home server will work alongside the home's router to accomplish this.

2.4.1 Web Server

The web application will be hosted in the HNS software's core system. A web server, Caddy, will handle HTTP or HTTPS requests from clients, and deliver the corresponding content. Specifically, Caddy will host the HNS web application via a reverse proxy. Other web applications managed by HNS may also be made available through reverse proxies.

2.4.2 Helper Program

For certain features, a helper program will be required on each device that the client wishes to use said features on. Notable features requiring a helper program are file synchronization, password management, and passkey management. This helper program will communicate with the home server to perform these features and assist with initial setup. Communication may be handled with TCP sockets to ensure data integrity while also having simple implementation.

2.4.3 File Synchronization

Before this feature is active, the user will have installed the helper program to each device they wish to synchronize files between. The user will have also selected folders on their client device to synchronize, and selected a corresponding folder on the home server. These should be done via the web application.

The master copies of files are stored on the home server. These files are then copied to client directories as needed. The helper program will observe changes in file data via checksum functions. When a change is found, a request to update the file is sent to the home server. Once verified by the server, the master copy is updated.

However, many devices may access the same file, and possibly make changes at the same time. If the home server were to receive multiple change requests for the same file, a merge conflict occurs. The system must receive user intervention, who will select which version of the file to keep. Optionally, the user may have adjusted a setting for the system to automatically override the need for user input. The setting can prefer the newer versus older version of a file, or the client versus server version.

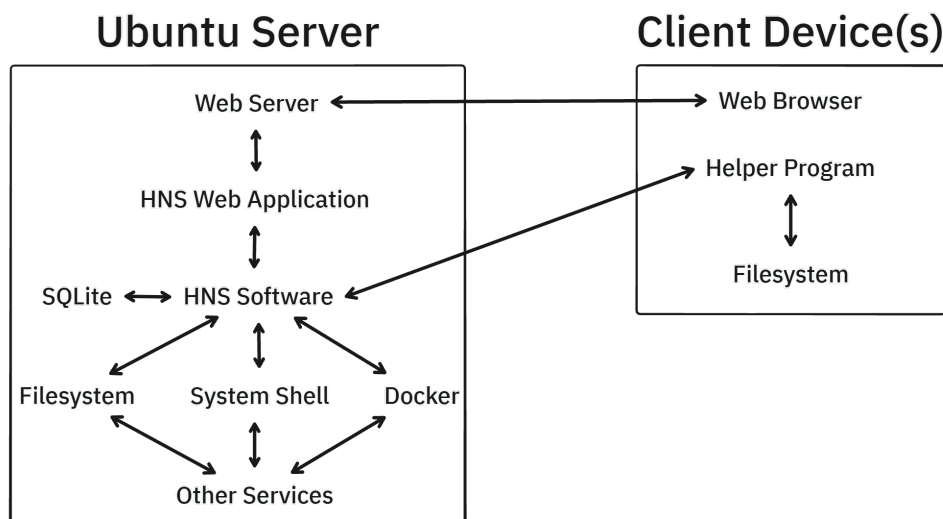
2.4.4 Password and Passkey Management

Similarly to the storage mechanisms in 2.4.1 File Synchronization, the home server will have a master copy of the user's passwords and passkeys. Whenever a password or passkey is needed by the user, the helper program will request the home server for a copy. The copy is then transferred to the client device for temporary use before being discarded.

3. System Architecture

3.1 Architecture Design

The Ubuntu home server hosts the web application, the SQLite database, and all background services. The user connects to the web application either locally via the local network or remotely via Tailscale, to access and manage the server.



3.2 Server Design

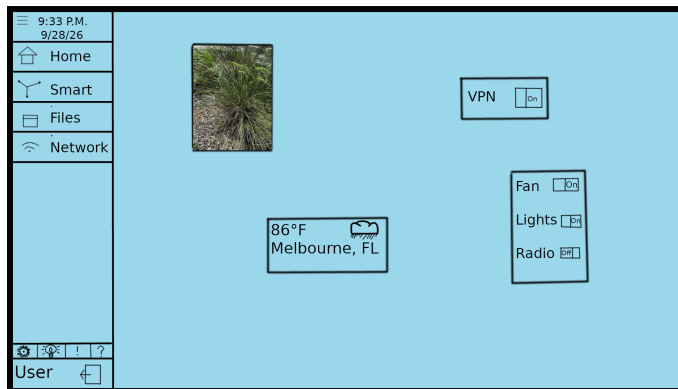
The HNS software will be designed to run on Ubuntu Server LTS 24.04. Most services will be run automatically on startup, including but not limited to: Pi-hole, Tailscale, Home Assistant, Jellyfin, and Caddy. Initial server setup will be handled by shell scripts which will install all required programs and configure them appropriately. Once this script is run, setup will continue from the web application, where the user will be able to add accounts.

The web application will be the primary method for the user to manage the system. The user may add, remove, or update services and tools from the server. These services are accessible through the same web application or by being connected to the home router.

3.3 Web Application Design

On the first launch, the website will redirect to a setup page, or a login page if already set up. The setup wizard will walk through any customization options for the user, as well as help them make an “administrator” account for the application (“administrator” referring to having control over the app, not over the system directly). Once setup is complete, the user will be redirected to the login page and, once logged in, redirected to the dashboard.

The dashboard page consists of two main parts: the sidebar menu and the widgets. The sidebar menu will be a collapsable menu acting almost as a table of contents for the website. This menu will contain a link to every category of features (e.g., “Smart Home,” “Files,” “Network,” etc.), as well as any pages for managing the website, such as “Settings” and “Account.”



The rest of the screen will be available for the user to place widgets. These widgets will allow the user to have certain settings available for a quick toggle or show information to them upon opening the app (e.g., local weather). The widgets will be freely movable and persistent. These widgets will be handled differently on mobile devices due to reduced screen real estate.

3.4 Design Rationale

The design for this server is meant to be simple, easy-to-learn, and reliable in order to minimize difficulties for the user. Ubuntu was chosen as it is one of the most popular consumer Linux distributions, and as such, any issues are likely very well documented, allowing for easier troubleshooting.

The website was designed with similar goals in mind. In order to not overwhelm the user, each page will only have a few effective types of buttons. For example, the dashboard (pictured in section 3.3) will have the left set of buttons, dedicated to reaching other pages, the account buttons (bottom-left), dedicated to the account and user preferences, and finally, the rest of the screen is dedicated to custom widgets added by the user, allowing them to only learn how to use what they want to use.

4. Data Design

4.1 Data Flow

The user data stays local to the home network. This allows the user to keep full control and ownership of their own data. All data flows entirely within the server. During authentication, the user ID and password entered are verified against the local SQLite database. A session token is then issued to the user. The configuration data includes network settings, hardware configuration, and the initial administrator account. This is written to the SQLite database with the optional apps reading the configuration on startup to launch properly.

The file synchronization feature uses TCP sockets between the client device and home server to manage file transfers. In cases where the device is connected remotely, Tailscale can be employed to ensure secure transmission of files. When a file is uploaded, it gets written to the server's filesystem with a record of the file's location, the uploader, a timestamp, and the type of file added. For downloads, the server retrieves the file's storage location from the database.

Widgets that display local network information get their data from the server. The weather widget is an example of an exception as it requires a connection to an external API. This request includes only necessary information and excludes any user or network related data. The analytics data is taken from the applications running on the server. This includes network activity logs, authentication attempts, and configuration changes. This data is aggregated locally and displayed on the Analytics page.

4.2 Data Description

SQLite is the chosen database because it doesn't require a separate server process and the performance matches the expected workload. Media files are not stored within the database itself, but instead on the server's filesystem. The database only stores the necessary metadata to locate and manage them. This keeps the database small and allows the storage capacity to be upgraded easily by adding more drives.

- The User Accounts contain the credentials and access level of each user authorized to connect to the web app. The fields used are the user ID, a username, a hashed password, and a role to distinguish admin accounts from standard accounts.
- Session Tokens contain the credentials and access level of each user authorized to connect to the web app. The fields are the token ID, the associated user ID, a creation timestamp, and an expiration timestamp.
- The Device Inventory shows all devices connected to the home network. The fields used are the device name, MAC address, IP address, and a last connected timestamp.
- The Service State records which applications are installed and are being actively used on the server.
- The Widget Configuration organizes each user's dashboard layout to match Section 3.3. The fields include a widget ID, widget type, screen position, and widget specific settings.
- The File Records store metadata for each file uploaded to the server. The fields include a file ID, file name, storage path, the uploader's user ID, the upload timestamp, file type, and the file size.
- The Analytics Records display timestamped events on the Analytics page. The fields include a timestamp of the event, an event type, the source application, and a description of the event.